

The Beginner's Essential Guide to Bash Scripting

Bash scripting turns repetitive manual terminal commands into fast, single-click automated tasks. Learning the basics saves hours of routine work and forms the foundation of modern DevOps.

What you'll learn:

- 1 Automating File Backups with Timestamps
- 2 Looping Through Multiple Servers or Files
- 3 Safe Script Execution with Error Handling
- 4 Accepting Dynamic Arguments from the Terminal

Aman Raj Singh

Cloud Operations Engineer @ iManage

linkedin.com/in/mramanrajsingh

1

TIP 1 OF 4

Automating File Backups with Timestamps

Manual backups are slow and prone to accidental overwrites. By combining the standard copy command with dynamic date subshells, you generate unique backup filenames instantly. This ensures you never overwrite yesterday's log file by accident. It is the perfect first practical script for any junior cloud engineer.

```
#!/bin/bash
BACKUP_DIR="./backups"
mkdir -p $BACKUP_DIR
cp app.log "$BACKUP_DIR/app_$(date +%Y-%m-%d).log"
echo "Backup complete!"
```

Cloud & DevOps Daily

Bash Scripting: Automate Repetitive Tasks

Tuesday, August 25, 2026

Swipe to tip 2 >> linkedin.com/in/mramanrajsingh

2

TIP 2 OF 4

Looping Through Multiple Servers or Files

Loops allow you to execute a command across dozens of targets automatically. Instead of checking servers one by one manually, a for loop iterates over a list in seconds. You define a variable, provide a list of items, and put your commands inside do...done. This eliminates repetitive typing and speeds up daily operational checks.

```
$#!/bin/bash
SERVERS=("192.168.1.10" "192.168.1.11" "192.168.1.12")
for ip in "${SERVERS[@]}"; do
    ping -c 1 "$ip" && echo "$ip is UP" || echo "$ip is DOWN"
done
```

Cloud & DevOps Daily

Bash Scripting: Automate Repetitive Tasks

Tuesday, August 25, 2026

Swipe to tip 3 >> linkedin.com/in/mramanrajsingh

3

TIP 3 OF 4

Safe Script Execution with Error Handling

Scripts execute line-by-line, and by default Bash keeps running even if a step fails. Adding basic error handling prevents dangerous automated chain reactions.

Using 'set -e' forces Bash to stop immediately if any command returns an error code.

This guarantees you won't deploy broken code or delete files when a build step fails.

```
#!/bin/bash
set -e # Exit immediately if any command fails

echo "Starting deployment..."
cd /var/www/myapp
git pull origin main
echo "Deployment successful!"
```

Cloud & DevOps Daily

Bash Scripting: Automate Repetitive Tasks

Tuesday, August 25, 2026

Swipe to tip 4 >> linkedin.com/in/mramanrajsingh

4

TIP 4 OF 4

Accepting Dynamic Arguments from the Terminal

Hardcoding values into your scripts makes them rigid and hard to reuse. Bash allows you to pass custom inputs when invoking your script using \$1, \$2, etc.

This makes your scripts flexible enough to reuse across dev, staging, and prod.

Always check if required arguments were actually provided before running core logic.

```
#!/bin/bash
ENV=$1

if [ -z "$ENV" ]; then
    echo "Error: Please specify an environment (dev/prod)."
```

Usage: ./deploy.sh dev"

```
    exit 1
fi

echo "Deploying application to $ENV environment..."
```


Cloud & DevOps Daily

Bash Scripting: Automate Repetitive Tasks

Tuesday, August 25, 2026

See Key Takeaways on next page >> linkedin.com/in/mramanrajsingh

Key Takeaways

- + Always put `#!/bin/bash` (shebang) on the very first line of your script.
- + Wrap variables in double quotes (`"$VAR"`) to prevent issues with spaces.
- + Make scripts executable using the command `chmod +x your_script.sh`.
- + Use descriptive variable names like `LOG_DIR` instead of single letters.
- + Test new scripts in a non-production folder before running live.

Watch Out For

- ! Adding spaces around equal signs when setting variables (e.g. `NAME = "Aman"` will fail).
- ! Forgetting to make the script executable before trying to run `./script.sh`.
- ! Unquoted path variables breaking scripts when directory names contain spaces.
- ! Not using `'set -e'`, allowing scripts to continue running after critical steps fail.

Follow for daily Cloud & DevOps guides!

linkedin.com/in/mrmanrajsingh